FPGA Implementation and Performance Analysis of Object Classification on a Video Stream

EC 491: UG PROJECT

MEMBERS

Anirudh Bhogi	-	22095011
Deepaprakash K	-	22095031
Pratyush Meshram	-	22095077
Rahul Sheokhand	-	22095084

Under the supervision of:

Dr K P Sarwadekar

DEPARTMENT OF ELECTRONICS ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY (BHU) VARANASI

CERTIFICATE

This is to certify that the UG Project entitled "**FPGA Implementation and Performance Analysis of Object Classification on a Video Stream**" submitted by Anirudh Bhogi (22095011), Deepaprakash K (22095031), Pratyush Meshram (22095077) and Rahul Sheokhand (22095084), to the Department of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University) Varanasi, in partial fulfilment of the requirements for the award of the degree "Bachelor of Technology" in Electronics Engineering is an authentic work carried out at Department of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University) Varanasi under my supervision and guidance on the concept video project grant as acknowledged.

Dr K P Sarwadekar

Associate Professor

Department of Electronics Engineering,

Indian Institute of Technology (BHU) Varanasi

DECLARATION

I hereby declare that the work presented in this project titled "**FPGA implementation and Performance Analysis of Object Classification on a Video Stream**" is an authentic record of our work carried out at the Department of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University), Varanasi as requirement for the award of degree of Bachelors of Technology in Electronics Engineering, submitted in the Indian Institute of Technology (Banaras Hindu University) Varanasi under the supervision of Dr K P Sarwadekar, Department of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University) Varanasi. It does not contain any part of the work, which has been submitted for the award of any degree either in this Institute or in other University/Deemed University without proper citation.

Anirudh Bhogi (22095011)

Deepaprakash K (22095031)

Pratyush Meshram (22095077)

Rahul Sheokhand (22095084)

ABSTRACT

This project presents an end-to-end pipeline for high-resolution aerial image classification, integrating custom deep-learning and traditional machine-learning techniques with hardware acceleration to meet the demands of edge and embedded computing environments. On the software side, a convolutional neural network (CNN) is designed from first principles in native C, implementing convolution, batch normalisation, ReLU activation, pooling, and fully connected layers. The CNN is optimised for 2K resolution grayscale imagery through algorithmic restructuring, loop unrolling, and memory-access tuning to minimise latency and maximise throughput. In parallel, a lightweight feature-based model using Histogram of Oriented Gradients (HOG) and an XGBoost classifier is developed to explore runtime-accuracy trade-offs and hybrid integration possibilities.

The hardware backend targets FPGA-based acceleration using the Xilinx ZCU104 board. The CNN is to be translated into a high-performance pipelined architecture via High-Level Synthesis (HLS) and RTL using Verilog, employing a 5-stage outer pipeline structure and leveraging fixed-point arithmetic for resource efficiency. A systolic array-based implementation of the convolution layers maximises parallelism, data reuse, and compute density by streaming pixel and weight data in orthogonal directions. Intermediate processing elements such as batch normalisation and pooling are restructured for DSP and LUT-friendly execution, ensuring tight timing closure and reduced logic utilisation.

Extensive benchmarking is to be performed against an NVIDIA Jetson Xavier GPU to compare latency, power, and classification accuracy under real-time constraints. This project thus establishes a scalable and portable foundation for high-efficiency aerial image inference pipelines, unifying software-based experimentation with hardware-level acceleration for real-world deployment in UAVs, surveillance, and smart city applications.

Contents

Chapter 1
INTRODUCTION

1.1. Background and Motivation

The increasing use of unmanned aerial vehicles (UAVs), especially drones, has created new challenges related to security, surveillance, and bird strikes at airports. For both civilian and military applications, it is essential to differentiate between drones and natural flying objects, such as birds. Traditional radar and motion-based techniques often fall short due to the similarities in their flight characteristics. This project presents a vision-based binary classification system designed to distinguish between drones and birds in grayscale aerial images. Using deep learning techniques developed in Python, the system is later converted to C and executed on hardware.

1.2. Problem Statement

The goal of this project is to build a robust, standalone binary classification system capable of accurately distinguishing between drones and birds in grayscale aerial images. The system is implemented entirely in native C & Verilog, without the use of any high-level libraries or external dependencies, ensuring maximum portability and compatibility with embedded platforms. Designed with efficiency in mind, the solution targets real-time performance in resource-constrained environments. Additionally, the architecture and implementation are tailored to be easily adaptable for hardware acceleration, with a future phase focusing on deployment to the Xilinx Zynq UltraScale+ ZCU104 FPGA board without an OS for optimised edge inference for 2k resolution and 60 Hz frame rate.

1.3. Objectives and Scope

This project focuses on designing a custom CNN architecture to classify grayscale aerial images of drones and birds, with the model implemented entirely in native C & Verilog to ensure compatibility with embedded systems and eliminate external dependencies. The pipeline includes preprocessing raw binary images and performing a forward pass using embedded pre-trained weights. An additional lightweight feature extraction module was explored, with its outputs evaluated using an XGBoost classifier as a sanity check for discriminative power. The entire system is designed to support future FPGA deployment, specifically for the ZCU104 board.

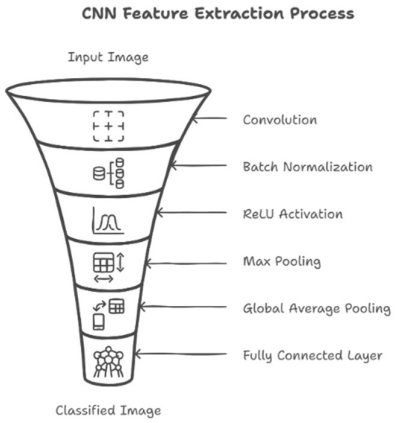
Chapter 2
DETAILED LITERATURE SURVEY OF EXISTING TECHNOLOGIES

2. 1. Existing Technologies and Challenges

Modern embedded inference solutions range from GPU-based SDKs to FPGA-centric flows, each with trade-offs. TensorFlow Lite for Microcontrollers provides a tiny C runtime for quantised models on microcontrollers, but demands heavy quantisation and limited operator support. ONNX Runtime enables portable inference across CPUs, GPUs, and accelerators, yet often requires per-backend tuning of memory layouts and graph optimisations. Xilinx Vitis AI automates quantisation and compilation for FPGA DPUs but introduces iterative model adjustments to fit device resources. High-level synthesis tools convert C/C++ kernels into FPGA logic and expose parallelism, yet achieving timing closure and efficient memory usage typically involves manual loop transformations and buffer partitioning. Traditional pipelines using HOG feature extraction coupled with an XGBoost classifier offer a lightweight, interpretable alternative, but handcrafted descriptors lack the representational power of deep networks and can become a bottleneck at high resolutions. Across all these approaches, core challenges include managing off-chip memory bandwidth for 2K images, mitigating accuracy loss from precision reduction, balancing FPGA resource constraints against performance targets, and navigating complex multi-step toolchains for model conversion and hardware build.

2.2. Rationale for our Approach

To address these challenges, we developed two standalone, dependency-free pipelines in native C on CPU platforms. First, we hand-implemented every CNN primitive— convolution, batch normalisation, activation, pooling, and fully connected layers— optimising memory access and maximising throughput on 2K resolution grayscale images. Second, we built an HOG extractor in C and integrated it with an XG-Boost classifier via its C API, demonstrating a fast, lightweight baseline for comparison. Both pipelines have been profiled and benchmarked to uncover insights into buffer management, precision trade-offs, and parallelism.

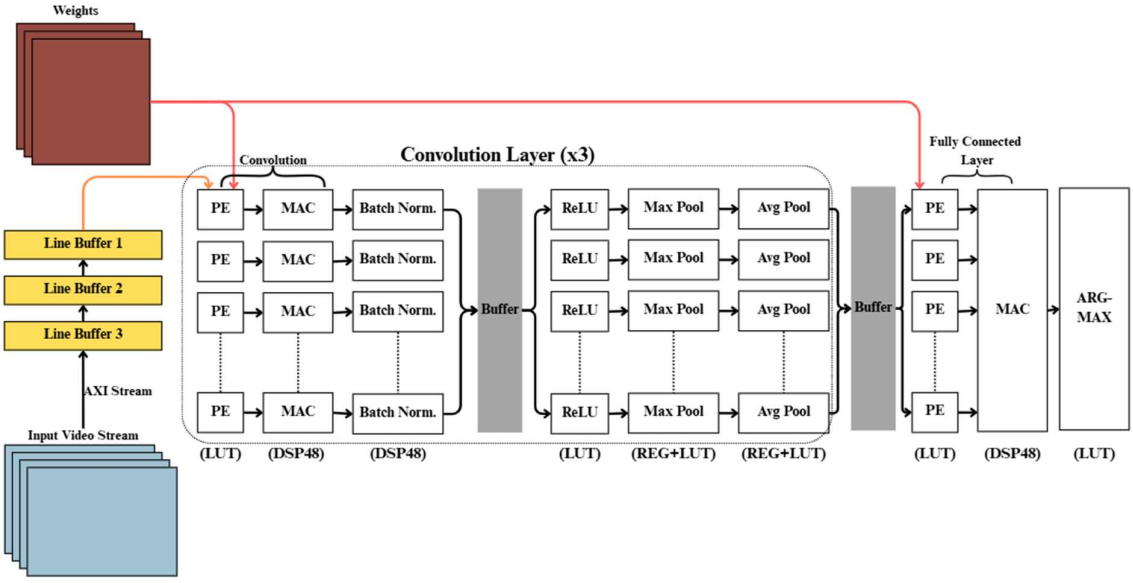


The above block diagram illustrates the architecture used for implementing the CNN-based classification pipeline in hardware. While software implementations on laptops leverage standard libraries such as OpenMP for optimisation, deploying a standalone system on an FPGA, without an OS, necessitates **bare-metal optimisation** techniques.

Synthesis on an FPGA can be achieved using traditional HDLs such as Verilog or via **High-Level Synthesis (HLS)**, which allows near-native C/C++ programs to be converted into RTL representations like Verilog or VHDL. The four primary hardware resources on an FPGA include **flip-flops**, **LUTs (Look-Up Tables)**, **BRAM (Block RAM)**, and **DSP48 slices** for arithmetic operations.

Zynq boards offer a heterogeneous architecture with a **Processing System (PS)**—typically an ARM Cortex-A9/A53 processor—and a **Programmable Logic (PL)** section, which is an Xilinx FPGA. Implementing the full model purely on PL is often inefficient due to its limited sequential control capabilities. Hence, a **PS+PL partitioned system** is used: the **PS** handles control-intensive sequential logic (like input handling and IP configuration), while the **PL** accelerates compute-heavy, parallelizable operations such as convolutions, pooling, and activation functions.

The PS is programmed using **C/C++**, typically via Xilinx SDK, to perform the system-level control, including memory-mapped I/O and interfacing with IPs. The PL is programmed using **Verilog or HLS-generated RTL**, implementing the custom CNN accelerator shown in the block diagram. An end-to-end embedded video processing system includes a **video input source** (e.g., camera, HDMI input, or pre-processed video frames from SD card), **frame buffering in DDR memory**, and a **pipelined CNN block** involving both **point processing** (like ReLU, Batch Norm) and **neighbourhood processing** (like Convolution, Max Pool). Finally, the processed output is sent to a **video output interface** such as HDMI, VGA, or DisplayPort.



The central aim of this project is to **accelerate the inference phase of a CNN-based image classifier on the PL**, and the block diagram below outlines the hardware implementation of a 3-layer CNN pipeline with convolution, batch normalization, ReLU, pooling, and a fully connected layer, efficiently mapped to FPGA resources like LUTs and DSPs.

The following justifies the approach illustrated in the block diagram:

• Input Video Stream

The input video stream can originate from various sources such as a camera feed following MIPI or UVC standards, an MP4 video file stored on an SD card, or even HDMI input from an external system like a laptop. This video data is processed as 24-bit RGB (8 bits per channel) with a Pixels-Per-Clock (PPC) value of 2, and it is transferred from memory via the AXI Stream interface.

• Weights

The pre-trained weights of the model are stored in the on-chip Block RAM (BRAM), with the Zynq UltraScale+ offering approximately 4 MB of BRAM. Assuming each weight parameter consumes a peak of 4 bytes, the memory required is calculated as:

$$Memory = 4 \times [(32 \times 1 \times 3 \times 3 + 32 + 32) + (64 \times 32 \times 3 \times 3 + 64 + 64) + (128 \times 64 \times 3 \times 3 + 128 + 128)] \approx \mathbf{500\ KB}$$

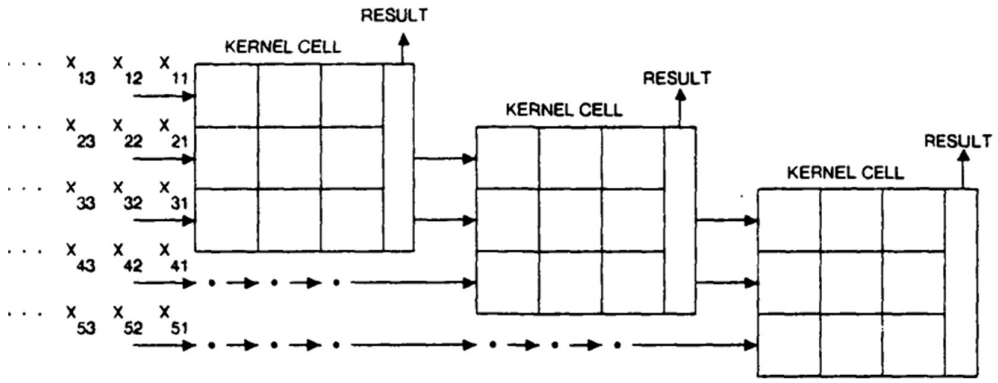
This implies that BRAM is sufficient to store all CNN weights, and due to the available capacity, array partitioning and loop unrolling techniques can be applied to enhance pipelining and parallelism.

• Line Buffers

Video frames are first sent to the DDR memory using Video Direct Memory Access (VDMA) for buffering, supporting up to 32 frame buffers. As the pixel data is read back via AXI Stream interface, line buffers are introduced to decouple memory latency. These are implemented using flip-flops or register arrays to enable low-latency access, allowing pipelined and parallel convolution operations by continuously feeding data to the processing elements.

• Convolution

The convolution block comprises an array of Processing Elements (PEs) and Multiply-Accumulate (MAC) units to perform 3×3 kernel-based convolution with a stride of 1. LUTs and DSP48 slices are utilised to implement the computational logic. To accelerate this process, a systolic array-based architecture is adopted. Systolic arrays offer significant benefits such as high throughput due to parallel execution, efficient weight reuse, and deep pipelining—each PE operates independently while passing intermediate results downstream. This architecture delivers better performance compared to traditional CPUs and even GPUs in certain use cases. The system performance will be validated by benchmarking classification throughput between the NVIDIA Jetson Xavier and the Zynq UltraScale+ platform.



- **Batch Normalisation**

Batch Normalisation typically involves division and square root operations, both of which are resource-intensive and consume a large number of LUTs. To optimise this for FPGA implementation, the expression is transformed into a Multiply-and-Accumulate (MAC)-friendly format using pre-computed constants:

$$y_i = \gamma \cdot \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad \rightarrow \quad y_i = x_i * inv\_stddev + shift$$

This reformulation replaces the division and square root with a multiplication and addition, enabling efficient implementation using DSP slices only, thus significantly reducing LUT usage.

• ReLU (Rectified Linear Unit)

$$ReLU(x) = \max(0, x)$$

This is a straightforward point-wise operation and can be implemented entirely using LUTs due to its simple conditional logic.

• Max Pool & Avg Pool

Both pooling operations are performed over a 2×2 window. Intermediate results are temporarily stored in registers, and the final pooling operation is computed using LUTs. These operations are time-multiplexed to run at one-fourth the operating frequency, which balances throughput and resource utilisation.

• Fully Connected Layer

This layer executes the final set of MAC operations, requiring the use of DSP blocks for high-speed multiply-accumulate functionality. The output is then passed to the *ArgMax* block, which identifies the class with the highest activation so it can be efficiently implemented using LUTs alone. The complete architecture forms a **5-stage outer pipeline**, comprising input loading, three convolutional layers, and final classification/post-processing. This pipelining maximises throughput by overlapping the execution of consecutive stages. Furthermore, floating-point operations from C/C++ are converted into fixed-point representation using 16-bit mantissa precision to align with the DSP block constraints, which support up to **24-bit × 18-bit** multiplications. This fixed-point optimisation ensures accurate yet efficient computation during inference on hardware

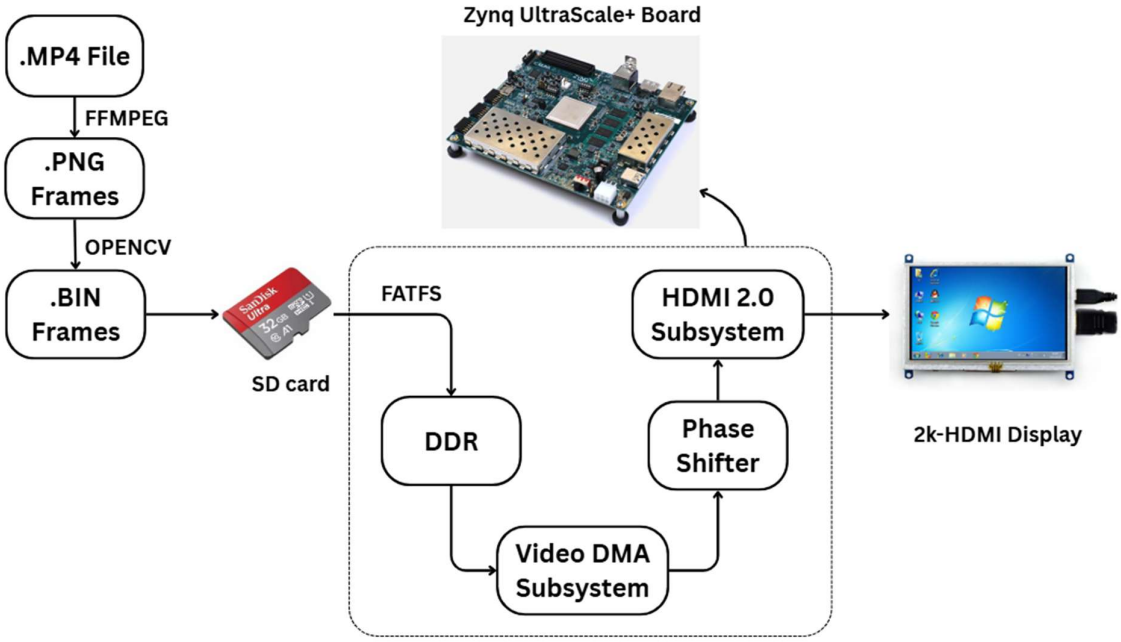
Chapter 3
WORK DONE

As explained in the approach, the task for building this Video Processing Embedded system can be divided roughly into the following tasks:

- Video Source Interfacing
- Video Processing Block Implementation
- Video Transmission and Streaming

3.1. Video Source Interfacing

The video source can be implemented in multiple ways; in our case, we opted to dump an MP4 file onto the SD card. Decoding an MP4 file requires a Video Decoder IP Core, which is a licensed IP from SoC Technologies and must be purchased for use. To avoid this, we used the Python library FFmpeg to extract individual PNG frames from the MP4 video at a frame rate of 60 fps. These PNG frames were then converted into BIN format using Opencv, and subsequently dumped onto the SD card.



Within the Xilinx SDK, by enabling the FAT File System, we were able to access the binary frames from the SD card and copy them into the DDR memory of the FPGA. The following block diagram illustrates the hardware setup for implementing the video streaming pipeline.

3.2. Video Processing Block Implementation

The video processing block implementation refers to deploying the CNN model or XGBoost for performing the required classification and displaying the output on an external display. As outlined in Section 2.2, both classification approaches have been carefully implemented in native C to ensure compatibility with hardware acceleration workflows and have shown satisfactory accuracy and inference times on large-scale datasets. The integration of this block into the overall system forms a critical bridge between algorithmic performance and practical deployment.

For the hardware implementation, we have planned two approaches: one using traditional HDL, such as Verilog, which is time-consuming in terms of implementation, and the other using High-Level Synthesis (HLS). This dual approach allows for a comparative analysis of performance, complexity, and resource management between the two implementations for object classification.

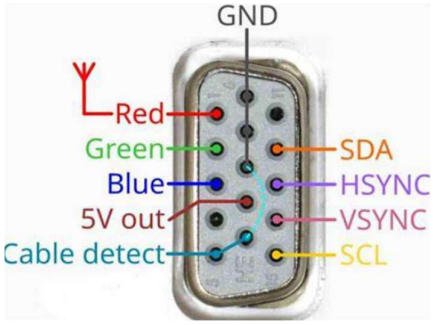
Regarding the Verilog implementation, the focus has primarily been on architectural research for designing the Convolutional Neural Network in hardware, along with resource estimation, which is discussed in detail in Section 2.2. The C implementation of the CNN model has been converted into an HLS-compatible version, but it remains in the debugging stage and is not yet fully developed.

3.3. Video Transmission and Streaming

Video Streaming could be done using various interfaces such as VGA (Video Graphics Array), HDMI (High-Definition Multimedia Interface) and DP (Display Port). Each comes with its pros and cons.

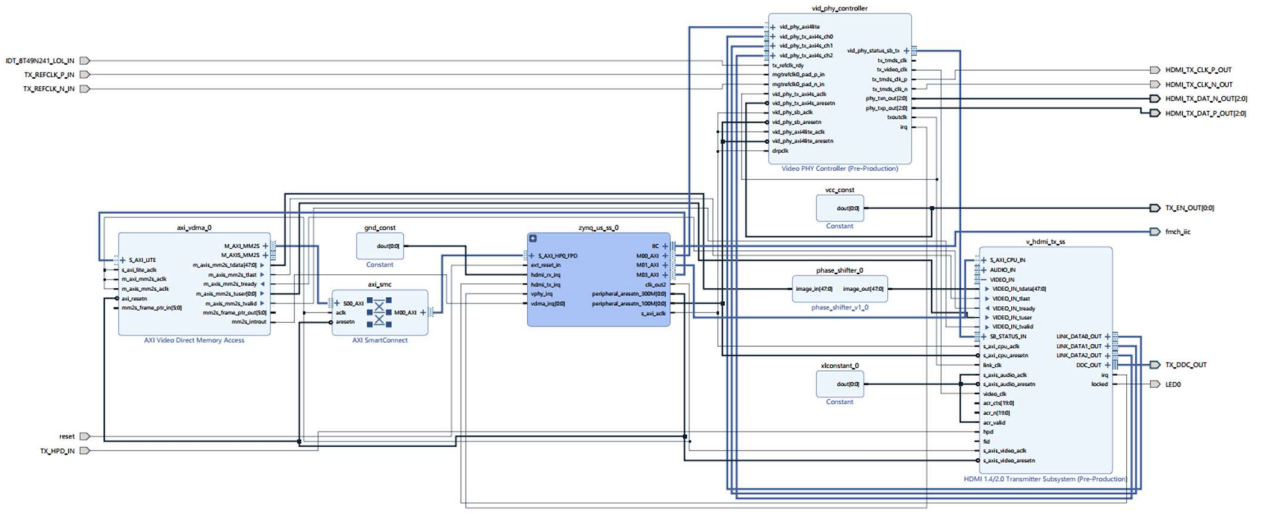
3.3.1. VGA Interface

The VGA interface is a simple interface consisting of basic synchronisation signals such as horizontal sync, vertical sync, RGB data, pixel clock, and cable detect. The primary issues with VGA are that it transmits RGB values as analog signals, making them highly susceptible to noise, and it cannot support high resolutions and high frame rates like 2K @ 60 fps, which is the target of the project. Therefore, we have transitioned from VGA to HDMI.

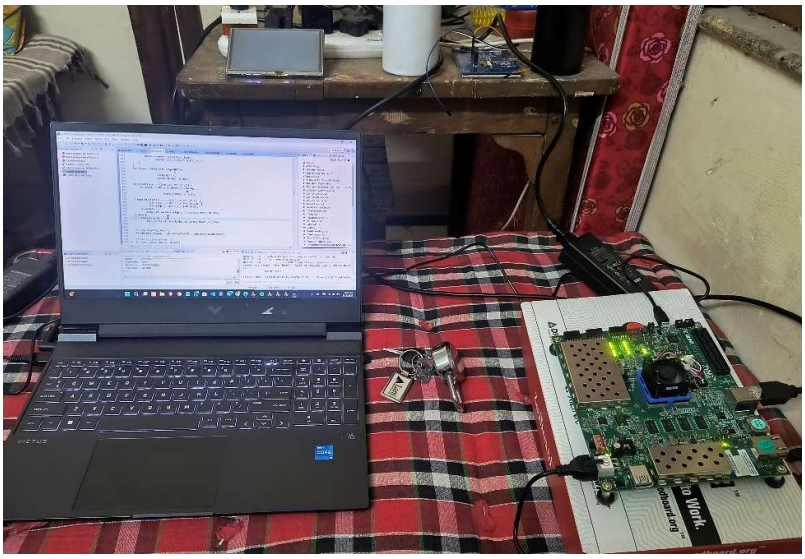


3.3.2. HDMI Interface

The Zynq UltraScale+ board includes a TI SN65DP159RGZ integrated circuit, which supports high-quality video streaming features such as 4K resolution, 120 fps, and 16-bit colour depth. These advanced capabilities come with the added complexity of HDMI integration, requiring additional IP cores—primarily the HDMI Tx Subsystem IP and the Video PHY Controller IP—for proper interfacing and functionality. The below picture shows the block diagram of HDMI Interfacing developed in Vivado 2018.3.



The HDMI Tx subsystem IP core is an IP core provided by Xilinx for generating TMDS (Transition Minimised Differential Signalling) encoded output on taking RGB/YUV input. It also comes with a DDC (Display Data Channel) support for communicating regarding the details of the display. The Video PHY Controller IP core is used to control the physical layer that is used to transmit the TMDS-encoded HDMI at high speeds via differential pairs.



The picture shown beside displays the Zynq UltraScale+ board, HDMI Display and other relevant hardware for video streaming. The DP port was not preferred due to its added complexity over HDMI and limited compatibility with most laptops lacking the DP port. Using HDMI, we have achieved the targeted 2k resolution video stream at 60 fps.

Chapter 4
EXPERIMENTS/SIMULATIONS AND RESULTS

4.1. Experiments/Simulations and Results

Hardware and Software Environment:

- **Processor:** Intel Core i5-11400H @ 2.7 GHz
- **Inference:** CPU only and no CUDA
- **Memory:** 16 GB RAM
- **Operating System:** Windows 11
- **Compiler:** GNU GCC C11 with optimization flags

Dataset and Preprocessing:

- **Image Resolution:** 2K (Grayscale)
- **Dataset Size:** 400 Bird and 428 Drone images with Binary Classification
- **Preprocessing:** Images converted to binary format with float32 pixel values

Evaluation Metrics:

- **Accuracy:** Correct classifications divided by total samples
- **Inference Time:** Average time to process a single image

4.2. Results

4.2.1 CNN Implementation (C Inference)

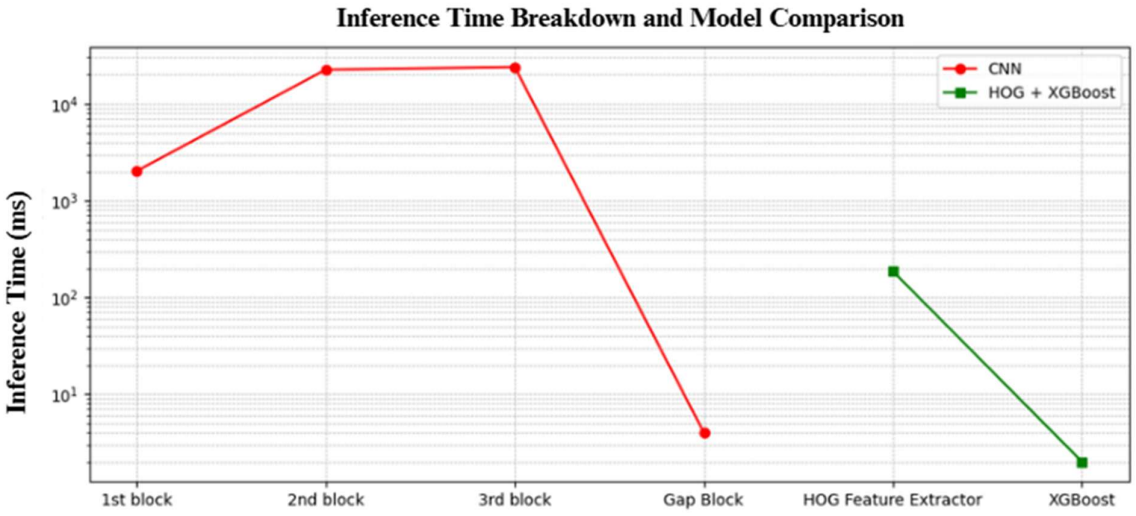
- **Accuracy:** 85% on the test dataset
- **Average Inference Time:** Approximately 45 seconds per 2K image

4.2.2 HOG + XGBoost Pipeline (C Implementation)

- **Accuracy:** 83.5% on the test dataset
- **Average Inference Time:** Approximately 180 milliseconds per 2048 × 2048 image

4.2.3 Comparative Analysis

Metric	Small CNN (C)	HOG + XGBoost (C)
Accuracy	85%	83.5%
Inference Time (s)	45 sec	180 ms



4.3. Summary

The CNN implementation achieved higher accuracy compared to the HOG + XGBoost pipeline, indicating better feature learning capabilities. However, it required **more computational resources** and **longer inference time**. The HOG + XGBoost approach was faster and more memory-efficient, but at the cost of reduced accuracy. These trade-offs highlight the considerations necessary when choosing between deep learning models and traditional machine learning pipelines for real-time image classification tasks. Both the approaches were successfully implemented and tested on high-resolution images. The CNN provided better accuracy, making it suitable for applications where precision is critical. In contrast, the HOG + XGBoost pipeline offered faster inference and lower resource consumption, which could be advantageous in resource-constrained environments. Future work will focus on hardware implementation using High-Level Synthesis (HLS) to further optimise performance and efficiency.

Chapter 5
CONCLUSION & FUTURE SCOPE

5.1. Challenges Faced

The challenges faced can be broadly divided into:

- Software Challenges
- Hardware Challenges

5.1.1. Software Challenges

In our software-only prototyping so far, we encountered several nontrivial challenges that shaped both our implementation strategy and our future FPGA roadmap:

1. **Manual Implementation of Every Layer**

   Writing convolution, batch-norm, ReLU, pooling, GAP, and fully-connected layers from scratch in C required careful attention to indexing, padding, and in-place updates. Without a high-level framework, even small mistakes in loop bounds or array strides could introduce subtle bugs that only manifested under certain image sizes or data layouts.

2. **Handling Very Large Inputs and Memory**

   Processing 2 K×2 K images as raw float32 arrays meant allocating tens of megabytes just for the input and intermediate feature maps. We had to pre-allocate large static buffers, carefully reuse them between layers, and avoid any hidden dynamic allocations or recursive calls, both to control peak RAM usage on the laptop and to keep the code HLS-friendly.

3. **Performance Profiling and Tuning**

   Early prototypes ran in tens of seconds per image. By instrumenting granular timing around each operation, we identified that the first convolution block and pooling steps were the biggest hotspots. That insight led us to tile loops for cache locality and fuse simple operations (e.g., bias addition + ReLU) to reduce memory traffic.

4. **Balancing Accuracy, Precision & Throughput**

   Working exclusively in float32 ensured numerical fidelity but also cost us performance. We experimented with reduced-precision accumulators and simple quantisation of weights to 16-bit or even 8-bit formats, carefully checking that the 85 % accuracy

baseline stayed within an acceptable ±1 % margin. These tests drove early decisions about fixed-point formats for the upcoming FPGA port.

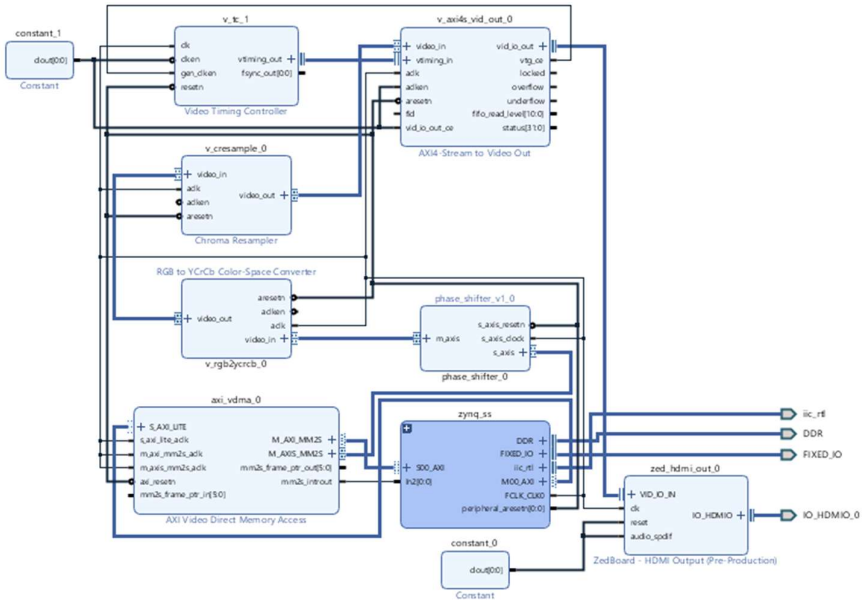
5. **Integrating HOG + XGBoost in C**

   Building our own HOG extractor in C—complete with gradient computation, orientation binning, and cell pooling—required similar buffer management and loop-level optimisations. Hooking its output into the XGBoost C API surfaced issues of data layout and type mismatches (double vs. float), which we resolved by standardising on float32 feature vectors and writing thin conversion wrappers.

6. **Ensuring HLS Compatibility**

   From day one, we structured every function to use only fixed-size arrays and very simple control flow (no recursion, no virtual calls, minimal branching) so that conversion via High-Level Synthesis tools will be straightforward. That constraint occasionally forced us to unroll loops by hand or avoid convenient dynamic data structures, trading off developer convenience for future hardware portability.

5.1.2. Hardware Challenges



One of the major hardware challenges encountered with the UltraScale+ board was the HDMI interfacing. HDMI interfacing is highly dependent on the specific hardware provided on the board. For instance, the Zynq Zedboard is equipped with an integrated circuit from Analog Devices, referred to as **ADV7511**. This IC is particularly useful because it can be directly interfaced with a lightweight IP core from Avnet by simply connecting its synchronisation signals from the Video Timing Controller and the RGB data from the VDMA. The image alongside illustrates a block diagram created for the HDMI interfacing of the Zedboard using the Avnet HDMI IP core. In contrast, the Pynq boards come with a dedicated IP core called RGB to DVI IP, which simplifies the HDMI interfacing process.

LUT	FF	BRAMs	URAM	DSP	LUTRAM	IO	GT
0	**0**	**0.00**	**0**	**0**	**0**	**9**	**0**
11830	18990	3.00	0	0	1332	11	3

LUT	FF	BRAMs	URAM	DSP	LUTRAM	IO	GT
0	**0**	**0.00**	**0**	**0**	**0**	**2**	**0**
3822	4934	3.50	0	5	446	23	0

The left-side image shows the resources consumed by **Zedboard for HDMI** interfacing, and the right-side image shows the resources consumed by the **UltraScale+ board for HDMI** interfacing. However, the maximum resolution and frame rate supported by the UltraScale+ board are way higher, but at the cost of extremely high complexity and resources.

cnn.cpp | top_level_csim.log

```
INFO: [SIM 2] *************** CSIM start ***************
INFO: [SIM 4] CSIM will launch GCC as the compiler.
   Compiling ../../../cnn_tb.cpp in release mode
   Compiling ../../../cnn.cpp in release mode
   Compiling ../../../model_weights.cpp in release mode
csim.mk:90: recipe for target 'obj/model_weights.o' failed
make: *** [obj/model_weights.o] Error 1
ERR: [SIM 100] CSim file generation failed: compilation error(s).
INFO: [SIM 3] *************** CSIM finish ***************

```

Another major issue being faced currently is in the implementation of the CNN model in HLS, in which the compilation fails due to an unknown reason, whose screenshot is shown below, over which debugging is currently still going on.

5.2. Conclusion

The project titled "**FPGA Implementation and Performance Analysis of Object Classification on Video Stream**" explores the application of various machine learning techniques—namely, a simple multi-layer Convolutional Neural Network (CNN) and a classical tree-based method, XGBoost—for real-time video stream classification. The focus is on hardware-centric implementation, utilising FPGA resources for deploying both CNN and XGBoost models.

The work involves designing a systolic array architecture to accelerate CNN inference on an FPGA and benchmarking its performance against GPU-based classification on the NVIDIA Jetson Xavier platform. A comprehensive analysis will be conducted to compare the advantages and limitations of FPGA-based and GPU-based solutions in terms of latency, throughput, power efficiency, and scalability.

Furthermore, the project evaluates the trade-offs between two hardware design methodologies: High-Level Synthesis (HLS) and Register-Transfer Level (RTL) HDL-based implementation, highlighting their impact on resource utilisation, performance, and development effort.

This project not only contributes a performance benchmark across heterogeneous computing platforms and design approaches, but also proposes the first known FPGA-based hardware

implementation for the IEEE Drone vs Bird Detection Challenge, making it a potentially valuable submission to the competition

5.3. Future Scope

The current implementation lays a solid foundation for deploying real-time object classification models on resource-constrained edge platforms. The insights and architectural frameworks developed during this project open multiple promising avenues for future work:

Support for Advanced Deep Learning Models

- Extend the architecture to support deeper CNNs (e.g., ResNet, MobileNet, YOLO-Tiny) optimised for FPGA.

Edge Deployment with On-Board Camera + Object Tracking

- Integrate the FPGA pipeline with live camera feeds (e.g., using MIPI or HDMI In).
- Extend the pipeline to include object tracking (e.g., Kalman filters, SORT/Deep SORT), making the solution more practical for real-time surveillance or drone navigation.

Low-Power ML Acceleration

- Perform detailed power-performance trade-off analysis, especially comparing FPGA vs Jetson Xavier under different workloads.
- Target ultra-low-power FPGA families (like Xilinx Artix-7 or Intel MAX10) for UAV-mounted inference systems

Scalable System Integration

- Combine the classification core with other subsystems, such as autonomous vehicles, which require efficiency and performance at the same time.

Open-Source Contribution & IEEE Challenge Impact

- Publish the architecture as an open-source baseline for hardware-aware object classification, specifically tailored for the IEEE Drone vs Bird Challenge.
- Set a precedent for hardware-accelerated solutions in drone detection, paving the way for future competitions and real-world deployments.

Chapter 6
BIBLIOGRAPHY

[1] Mittal, Sparsh. "A survey of FPGA-based accelerators for convolutional neural networks." *Neural computing and applications* 32, no. 4 (2020): 1109-1139.

[2] Kung, H.T., 1982. *Why systolic architecture?* (pp. 37-46). Design Research Center, Carnegie-Mellon University.

[3] Kung, H.T. and Song, S.W., 1981. A systolic 2-D convolution chip. *COMPUTER SCIENCE*, *3*, p.0198.

[4] Wei, X., Yu, C.H., Zhang, P., Chen, Y., Wang, Y., Hu, H., Liang, Y. and Cong, J., 2017, June. Automated systolic array architecture synthesis for high-throughput CNN inference on FPGAs. In *Proceedings of the 54th Annual Design Automation Conference 2017* (pp. 1-6).

[5] Chua, S.H., Teo, T.H., Tiruye, M.A. and Wey, I.C., 2022, December. Systolic array-based convolutional neural network inference on FPGA. In *2022 IEEE 15th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)* (pp. 128-133). IEEE.

[6] *Video PHY Controller Product Guide* (PG230) [By Xilinx]

[7] *HDMI 1.4/2.0 TX Subsystem Product Guide* (PG235) [By Xilinx]